

Corporate Equipment Allocation and Tracking System

1. Introduction	1
2. Problem Statement	2
3. Architecture Overview	2
4. Component Design	4
5. System Workflow	5
6. Microservices Design	6
7. REST API Endpoints	7
7.1 API Request & Response Examples	7
Authentication	7
User Management	9
Create User	9
Get All Users	11
Get User by ID	12
Update User	13
Equipment Management	14
Create Equipment	14
Update Equipment	16
List Equipment:	18
Mark Damaged Equipment	18
Allocation and Return APIs	19
Allocate Equipment	19
Return Equipment	23
Get Allocation Status by User	26
7.2 Notification Service	26
8.Database Design	28
9. Security	31
Authorization: Role-based access control.	31
10. Key Design Decisions	34
11. Conclusion	34

1. Introduction

The Corporate Equipment Allocation and Tracking System is designed to streamline the distribution and tracking of office equipment, such as laptops, projectors, and tablets. Reception staff can efficiently allocate equipment, track its availability, and ensure timely returns. The system also automates notifications to employees, maintenance, and inventory teams, improving operational efficiency and accountability.

2. Problem Statement

Managing office equipment manually can lead to delays, misplaced items, and missed maintenance requirements. Reception staff need a system to:

- Record equipment allocations and returns.
- Track employee usage and expected return dates.
- Automate reminders for overdue returns.
- Alert maintenance for damaged equipment.
- Keep inventory records up-to-date.

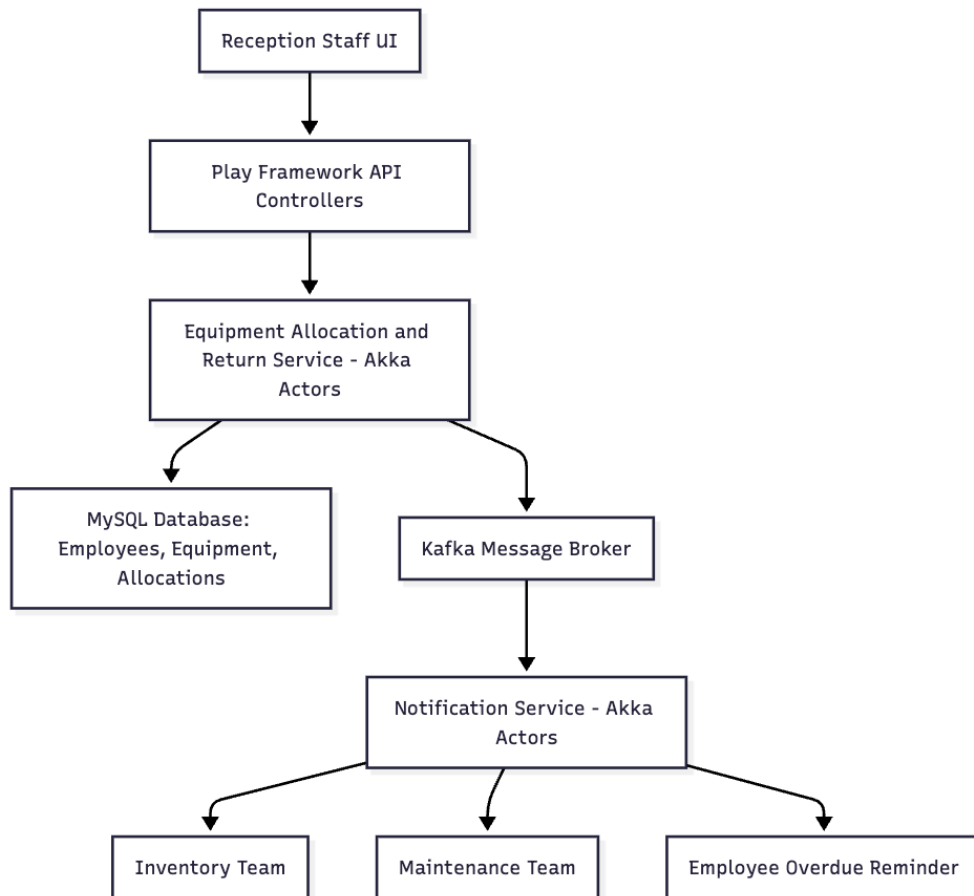
This system aims to solve these challenges by providing a reliable, scalable, and automated solution.

3. Architecture Overview

The system follows a **modular microservice-based architecture**. The key components include:

1. **Frontend / Application Layer** – Interfaces for reception staff to request allocation/return and view equipment status.
2. **API Layer (Play Framework)** – Handles REST requests for allocation, return, and status updates.
3. **Service Layer** – Contains business logic, validation, and workflows.
4. **Database Layer (MySQL)** – Stores employees, equipment, and allocation records.

5. **Notification Service (Akka Actors)** – Handles asynchronous notifications like overdue reminders and maintenance alerts.
6. **Message Broker (Kafka)** – Queues notifications to ensure reliable delivery.
7. **Deployment** – Docker containers for consistent and scalable deployment.



4. Component Design

Frontend / App:

- User interface for reception staff.
- Submit allocation and return requests.
- View status of equipment.

Play API Controllers:

- Receive and validate REST API requests.
- Pass data to the Service Layer.

Service Layer:

- Apply business logic for allocation and return.
- Trigger notifications based on events (overdue, damage, etc.).

Database (MySQL):

- Tables: Employees, Equipment, Allocations.
- Stores allocation history and status.

Akka Notification Service:

- Actors manage asynchronous notifications.
- Schedulers handle reminders for overdue equipment.
- Kafka consumers listen for events from the database/service layer.

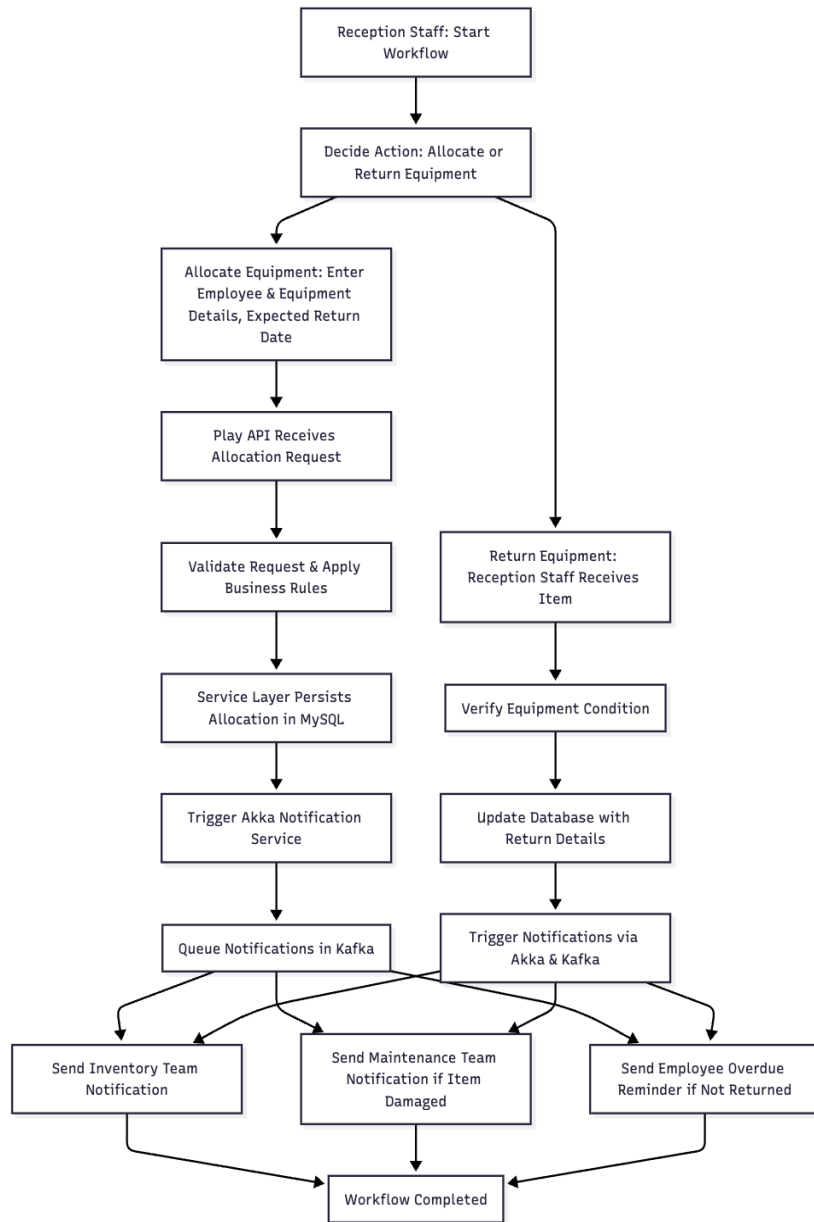
Kafka (Messaging Layer):

- Queue notifications for delivery to email services.
- Ensures no notifications are lost even if services are busy.

Docker Deployment:

- Each component runs in its own container for consistency.
- Supports scalability and easy environment replication.

5. System Workflow



6. Microservices Design

The system implements a microservice architecture for modularity and scalability.

Microservices and Purpose

- Equipment Allocation Service:
 - Handles allocation, return, and equipment status tracking.
 - Updates the database with allocation and return information.
 - Applies business rules to ensure equipment availability.
- Notification Service:
 - Sends reminders to employees for overdue equipment.
 - Alerts maintenance for damaged items.
 - Notifies inventory about allocations and returns.

Inter-Service Communication

- Synchronous: Frontend → Equipment Allocation Service via REST API.
- Asynchronous: Equipment Allocation Service → Notification Service via Kafka.

Design Rationale:

- Separating notification logic from allocation ensures high responsiveness.
- Independent scaling is possible for services under heavy load.
- Improves maintainability and modularity.

7. REST API Endpoints

The Corporate Equipment Allocation and Tracking System exposes the following REST APIs to manage equipment allocation, return, and status tracking. These endpoints are primarily consumed by the frontend application used by reception staff.

7.1 API Request & Response Examples

Authentication

1. Login

URL: POST — <http://localhost:9000/api/login>

Input:

```
{  
  "username": "admin",  
  "password": "Admin@123"  
}
```

Output:

```
{  
  "status": "success",  
  "message": "Login successful",  
  "data": {  
    "token":  
    "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyYb2xlIjoiYWRtaW4iLCJleHAiOiJlE3NjM4MjY0MzMsInVzZXIiOiJhZG1pb2IiIm1hdCI6MTc2MzgyMjgzM30.YwsJ5pb0BVWPtsra7XdgFoju8RJ8iMrih8_zzPsb2WA",  
    "expires_in": 3600  
  }  
}
```

Case fail:

POST http://localhost:9000/api/login

Docs Params Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   {
3     "username": "admin",
4     "password": "Admin@12322"
5   }
6 }
```

Body Cookies Headers (4) Test Results (0/1) 401 Unauthorized 209 ms 1

{ } JSON Preview Debug with AI

```
1 {
2   "status": "fail",
3   "message": "Invalid username or password"
4 }
```

Case Success:

POST http://localhost:9000/api/login

Docs Params Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   {
3     "username": "admin",
4     "password": "Admin@123"
5   }
6 }
```

Body Cookies Headers (4) Test Results (1/1) 200 OK 312 ms 378 B

{ } JSON Preview Visualize

```
1 {
2   "status": "success",
3   "message": "Login successful",
4   "data": {
5     "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyYb2x1IjoieWRtaW4iLCJleHAiOjE3NjM4MjIzOTgsInVzZXIiOiJhZG1pb2IiLCJhdCI6MTc2MzgxODc5OH0.MQ8U7VeJT-vMKck9SZPXsvQncHicWaImVwU8lig0TAQ",
6     "expires_in": 3600
7   }
8 }
```


User Management

Create User

URL: POST <http://localhost:9000/api/users/createUser>

Input:

```
{  
  "username": "Manikanta",  
  "password": "Manikanta@123",  
  "role": "user",  
  "name": "Manikanta",  
  "department": "IT",  
  "email": "manikanta@gmail.com"  
}
```

OUTPUTS:

```
{  
  "status": "fail",  
  "message": "Username 'Manikanta' already exists"  
}  
  
{  
  "status": "success",  
  "message": "User created successfully",  
  "data": {  
    "id": 15,  
    "username": "Srikanath",  
    "password": "Srikanth@123",  
    "role": "user",  
    "name": "Srikanath",  
    "department": "IT",  
    "email": "srikanath12@gmail.com"  
  }  
}
```

POST http://localhost:9000/api/users/createUser

Docs Params Authorization Headers (10) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```

1 {
2   "username": "Manikanta",
3   "password": "Manikanta@123",
4   "role": "user",
5   "name": "Manikanta",
6   "department": "IT",
7   "email": "manikanta@gmail.com"
8 }
9

```

Body Cookies Headers (4) Test Results 200 OK

{ JSON Preview Visualize

```

1 {
2   "status": "fail",
3   "message": "Username 'Manikanta' already exists"
4 }

```

POST http://localhost:9000/api/users/createUser

Docs Params Authorization Headers (10) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```

1 {
2   "username": "Srikanath",
3   "password": "Srikanth@123",
4   "role": "user",
5   "name": "Srikanath",
6   "department": "IT",
7   "email": "srikanath12@gmail.com"
8 }
9

```

Body Cookies Headers (4) Test Results 200 OK

{ JSON Preview Visualize

```

1 {
2   "status": "success",
3   "message": "User created successfully",
4   "data": {
5     "id": 15,
6     "username": "Srikanath",
7     "password": "Srikanth@123",
8     "role": "user",
9     "name": "Srikanath",
10    "department": "IT",
11    "email": "srikanath12@gmail.com"
12  }
13 }

```

Get All Users

URL: GET http://localhost:9000/api/users

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:9000/api/users
- Headers (7):** Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyYb...
- Status:** 200 OK
- Body (JSON):**

```
{
  "status": "success",
  "message": "Users fetched successfully",
  "data": [
    {
      "id": 1,
      "username": "admin",
      "password": "Admin@123",
      "role": "admin",
      "name": "Naresh Nagula",
      "department": "IT",
      "email": "naresh996463@gmail.com"
    },
    {
      "id": 2,
      "username": "Hardhik",
      "password": "Hardhik@123",
      "role": "user",
      "name": "Hardhik",
      "department": "IT",
      "email": "naresh119935@gmail.com"
    }
  ]
}
```

Get User by ID

URL: GET http://localhost:9000/api/users/3

GET

http://localhost:9000/api/users/3

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Headers

6 hidden

	Key	Value	Description
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyb...	
	Key	Value	Description

Body

Cookies

Headers (4)

Test Results

200 OK

JSON

Preview

Visualize

```
1 {
2   "status": "success",
3   "message": "User fetched successfully",
4   "data": {
5     "id": 3,
6     "username": "reception1",
7     "password": "Recep@123",
8     "role": "reception",
9     "name": "Reception Staff",
10    "department": "Front Desk",
11    "email": "reception@example.com"
12  }
13 }
```

Update User

URL: PUT <http://localhost:9000/api/users/15>

PUT <http://localhost:9000/api/users/15>

Docs Params Authorization Headers (9) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON**

```
1 {
2   |   "name": "Srikanath Nair",
3   |   "department": "BP0"
4   | }
```

Body Cookies Headers (4) Test Results

200 OK

{} JSON Preview Visualize

```
1 {
2   |   "status": "success",
3   |   "message": "User updated successfully"
4   | }
```

Delete User

URL: PUT <http://localhost:9000/api/users/15>

DELETE <http://localhost:9000/api/users/16>

Docs Params **Authorization** Headers (7) Body Scripts Settings

Headers 6 hidden

	Key	Value	Description
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyY...	
	Key	Value	Description

Body Cookies Headers (4) Test Results

200 OK

{} JSON Preview Visualize

```
1 {
2   |   "status": "success",
3   |   "message": "User deleted successfully"
4   | }
```

Equipment Management

Create Equipment

- URL: POST — <http://localhost:9000/api/equipment/create>
- Purpose: Created equipment details

Input:

```
{  
  "name": "MSI Titan",  
  "type": "Laptop"  
}
```

Output

```
{  
  "status": "success",  
  "message": "Equipment created successfully",  
  "data": {  
    "equipment": {  
      "id": 14,  
      "name": "MSI Titan Xpro",  
      "type": "Laptop",  
      "status": "available"  
    }  
  }  
}
```

Case fail:

POSThttp://localhost:9000/api/equipment/create

DocsParamsAuthorizationHeaders (10)BodyScriptsSettings

noneform-datax-www-form-urlencodedrawbinaryGraphQLJSON

```
1 {
2   "name": "MSI Titan",
3   "type": "Laptop"
4 }
5
```

BodyCookiesHeaders (4)Test Results

200 OK

{ }JSONPreviewVisualize

```
1 {
2   "status": "fail",
3   "message": "Equipment already exists",
4   "data": {}
5 }
```

POSThttp://localhost:9000/api/equipment/create

DocsParamsAuthorizationHeaders (10)BodyScriptsSettings

Headers8 hidden

	Key	Value	Description	Bulk
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJy...		
☒	Content-Type	application/json		
	Key	Value	Description	

BodyCookiesHeaders (4)Test Results

401 Unauthorized19 ms

{ }JSONPreviewDebug with AI

```
1 {
2   "status": "fail",
3   "message": "Invalid or expired token"
4 }
```

Case Success:

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** `http://localhost:9000/api/equipment/create`
- Body:**

```
{
  "name": "MSI Titan",
  "type": "Laptop"
}
```
- Response:** 200 OK
- Body (JSON):**

```
{
  "status": "success",
  "message": "Equipment created successfully",
  "data": {
    "equipment": {
      "id": 13,
      "name": "MSI Titan",
      "type": "Laptop",
      "status": "available"
    }
  }
}
```

Update Equipment

URL: PUT — <http://localhost:9000/api/equipment/update>

Input:

```
{
  "id": 13,
  "name": "MSI Titan",
  "type": "Laptop",
  "status": "allocated"
}
```


Output:

```
{
  "status": "success",
  "message": "Equipment updated successfully",
  "data": {
    "equipment": {
      "id": 13,
      "name": "MSI Titan",
      "type": "Laptop",
      "status": "allocated"
    }
  }
}
```

PUT

http://localhost:9000/api/equipment/update

Docs

Params

Authorization

Headers (9)

Body

Scripts

Settings

Headers

8 hidden

	Key	Value	Description
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJy...	
	Key	Value	Description

Body

Cookies

Headers (4)

Test Results

200 OK

{}

JSON

Preview

Visualize

```
1  {
2    "status": "success",
3    "message": "Equipment updated successfully",
4    "data": {
5      "equipment": {
6        "id": 13,
7        "name": "MSI Titan",
8        "type": "Laptop",
9        "status": "allocated"
10     }
11   }
12 }
```

List Equipment:

URL: GET <http://localhost:9000/api/equipment/list>

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** <http://localhost:9000/api/equipment/list>
- Headers (7):** Includes an Authorization header with a Bearer token.
- Status:** 200 OK
- Body (JSON):**

```
1 {
2   "status": "success",
3   "message": "Equipment list fetched",
4   "data": [
5     {
6       "id": 1,
7       "name": "hp laptop",
8       "type": "Laptop",
9       "status": "allocated"
10    },
11    {
12      "id": 2,
13      "name": "apple laptop",
14      "type": "Laptop",
15      "status": "allocated"
16    },
17    {
18      "id": 3,
19      "name": "Asus laptop",
20      "type": "Laptop",
21      "status": "allocated"
22    }
23  ]
24 }
```

Mark Damaged Equipment

URL: PUT <http://localhost:9000/api/equipment/mark-damaged/:equipmentId>

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** <http://localhost:9000/api/equipment/mark-damaged/11>
- Headers (8):** Includes an Authorization header with a Bearer token.
- Status:** 200 OK
- Body (JSON):**

```
1 {
2   "status": "success",
3   "message": "Equipment marked as damaged",
4   "data": {
5     "equipment": {
6       "id": 11,
7       "name": "Lenovo",
8       "type": "Tablets",
9       "status": "damaged"
10    }
11  }
12 }
```

Allocation and Return APIs

Allocate Equipment

URL: POST <http://localhost:9000/api/equipment/allocate>

Behaviour:

Success:

Creates allocation. Equipment marked allocated. Kafka event sent.

Fail:

Invalid body, equipment already allocated, or missing users.

Input:

```
{
  "userId": 14,
  "equipmentId": 11,
  "expectedReturn": "2025-11-20 18:00:00"
}
```

Case Fail:

POST <http://localhost:9000/api/equipment/allocate>

Docs Params Authorization Headers (11) Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "userId": 14,
3   "equipmentId": 11,
4   "expectedReturn": "2025-11-20 18:00:00"
5 }
```

Body Cookies Headers (4) Test Results 200 OK

{ } JSON Preview Visualize

```
1 {
2   "status": "fail",
3   "message": "Equipment is already allocated",
4   "data": {}
5 }
```

Case Success:

```
{
  "status": "success",
  "message": "Equipment allocated successfully",
  "data": {
    "allocation": {
      "equipment": {
        "id": 11,
        "name": "Lenovo",
        "type": "Tablets",
        "status": "allocated"
      },
      "employee": {
        "id": 14,
        "name": "Manikanta",
        "department": "IT"
      },
      "allocatedAt": "2025-11-22 20:22:37.347",
      "expectedReturn": "2025-11-20 18:00:00.0"
    }
  }
}
```

POST http://localhost:9000/api/equipment/allocate

Docs Params Authorization Headers (11) Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "userId": 14,
3   "equipmentId": 11,
4   "expectedReturn": "2025-11-20 18:00:00"
5 }
6
```

Body Cookies Headers (4) Test Results 200 OK

{ } JSON Preview Visualize

```
1 {
2   "status": "success",
3   "message": "Equipment allocated successfully",
4   "data": {
5     "allocation": {
6       "equipment": {
7         "id": 11,
8         "name": "Lenovo",
9         "type": "Tablets",
10        "status": "damaged"
11      },
12      "employee": {
13        "id": 14,
14        "name": "Manikanta",
15        "department": "IT"
16      },
17      "allocatedAt": "2025-11-22 20:19:58.835",
18      "expectedReturn": "2025-11-20 18:00:00.0"
19    }
20  }
21 }
```

Kafka notification triggered post-allocation for employee and inventory teams.

Employee

← → ↺ localhost:8025

 MailHog

←

🗑️

⬇️

🔄

🟢 Connected

Inbox (16)

⊗ Delete all messages

Jim

Jim is a chaos monkey.
[Find out more at GitHub.](#)

Enable Jim

Plain text Source

From noreply@company.com
Subject **Equipment Allocated**
To manikanta@gmail.com

Dear Employee,

Equipment ID 11 has been allocated to you.

Thank you.

Inventory teams

← → ↺ localhost:8025

 MailHog

←

🗑️

⬇️

🔄

🟢 Connected

Inbox (16)

⊗ Delete all messages

Jim

Jim is a chaos monkey.
[Find out more at GitHub.](#)

Enable Jim

Plain text Source

From noreply@company.com
Subject **Equipment Allocated (ID: 11)**
To nareshnagula61@gmail.com

Dear Inventory Team,

Equipment ID 11 has been allocated.

Thank you.

Return Equipment

URL: POST <http://localhost:9000/api/equipment/return>

Behaviour:

Success:

Updates allocation. Equipment status updated. Kafka events triggered.

Fail:

Invalid body, no active allocation, or missing users.

The screenshot displays a REST client interface with the following details:

- Method:** POST
- URL:** <http://localhost:9000/api/equipment/return>
- Body:** Raw JSON format with the following content:

```
1 {
2   "equipmentId": 11,
3   "userId": 14,
4   "condition": "Good"
5 }
```
- Response:** 200 OK
- Response Body:** JSON format with the following content:

```
1 {
2   "status": "success",
3   "message": "Equipment returned successfully",
4   "data": {
5     "allocation": {
6       "equipment": {
7         "id": 11,
8         "name": "Lenovo",
9         "type": "Tablets",
10        "status": "allocated"
11      },
12      "employee": {
13        "id": 14,
14        "name": "Manikanta",
15        "department": "IT"
16      },
17      "allocatedAt": "2025-11-22 20:22:37.0",
18      "expectedReturn": "2025-11-20 18:00:00.0",
19      "returnedAt": "2025-11-22 20:39:06.404",
20      "equipmentCondition": "Good"
21    }
22  }
23 }
```

Kafka notification triggered post-allocation for employee and inventory teams.

employee:

 MailHog

←

🗑️

⬇️

🔄

🟢 Connected

Inbox (26)

⊗ Delete all messages

Jim

Jim is a chaos monkey.
[Find out more at GitHub.](#)

Enable Jim

Plain text

Source

From noreply@company.com

Subject **Equipment Returned**

To manikanta@gmail.com

Dear Employee,

Equipment ID 11 has been returned.

Condition: OK

Thank you.

Inventory teams

 MailHog

←

🗑️

⬇️

🔄

🟢 Connected

Inbox (26)

⊗ Delete all messages

Jim

Jim is a chaos monkey.
[Find out more at GitHub.](#)

Enable Jim

Plain text

Source

From noreply@company.com

Subject **Equipment Returned (ID: 11)**

To nareshnagula61@gmail.com

Dear Inventory Team,

Equipment ID 11 has been returned.

Condition: OK

Thank you.

Fail case:

POST

http://localhost:9000/api/equipment/return

Docs

Params

Authorization

Headers (9)

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

1

{

2

"equipmentId": 11,

3

"userId": 14,

4

"condition": "Good"

5

}

Body

Cookies

Headers (4)

Test Results

200 OK

{}

JSON

Preview

Visualize

1

{

2

"status": "fail",

3

"message": "No active allocation found for this employee",

4

"data": {}

5

}

POST

http://localhost:9000/api/equipment/return

Docs

Params

Authorization

Headers (9)

Body

Scripts

Settings

Headers

8 hidden

	Key	Value	Description
☐	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyYb...	
	Key	Value	Description

Body

Cookies

Headers (4)

Test Results

401 Unauthorized

{}

JSON

Preview

Debug with AI

1

{

2

"status": "fail",

3

"message": "Missing Authorization header"

4

}

Get Allocation Status by User

URL: GET <http://localhost:9000/api/equipment/status/:userId>

Behaviour:

Success:

Returns allocation list or empty list.

Fail:

Unauthorized.

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** `http://localhost:9000/api/equipment/status/14`
- Headers:** 7 headers, including an Authorization header with a Bearer token.
- Status:** 200 OK
- Body:** JSON response showing allocation status.

```
1 {
2   "status": "success",
3   "message": "Allocation status fetched",
4   "data": [
5     {
6       "equipment": {
7         "id": 10,
8         "name": "Lenovo",
9         "type": "Projector",
10        "status": "available"
11      },
12      "employee": {
13        "id": 14,
14        "name": "Manikanta",
15        "department": "IT"
16      },
17      "allocatedAt": "2025-11-19 22:45:46.0",
18      "expectedReturn": "2025-11-20 18:00:00.0",
19      "returnedAt": "2025-11-19 20:28:37.0",
20      "equipmentCondition": "Damaged"
21    },
22  ]
23 }
```

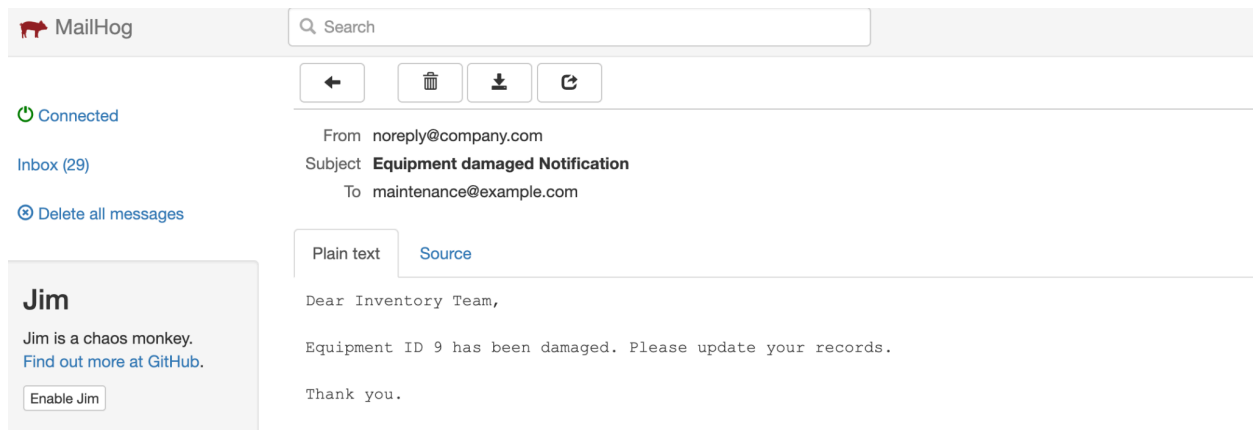
7.2 Notification Service

- **Purpose:** Handle all automated notifications related to equipment allocation and return.
- **Endpoints:** No public REST endpoints.
- **Behavior:**

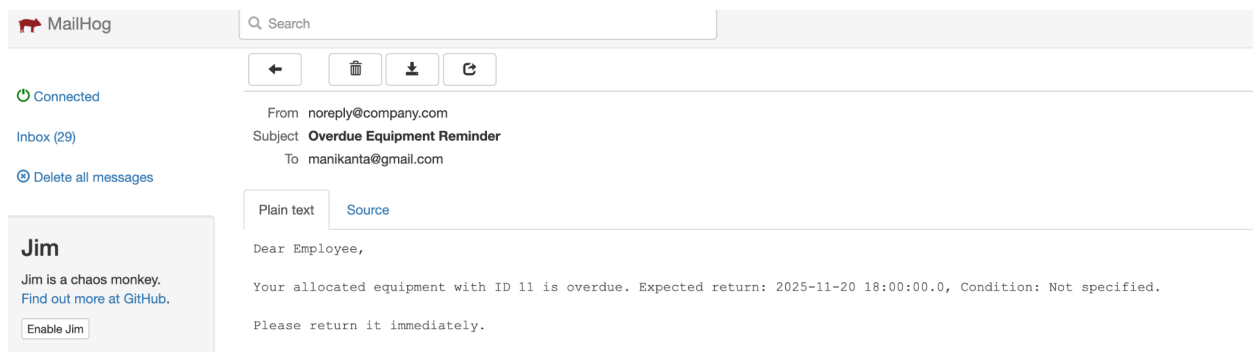
- Consumes events from Kafka triggered by allocation or return actions.
- Sends notifications via email/SMS to employees, inventory, and maintenance teams.

Ensures asynchronous, reliable delivery of reminders and alerts.

Notification sent for damaged equipment



Notification sent for overdue reminder



8.Database Design

DataBase name--naresh

```
CREATE TABLE users (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(50) UNIQUE NOT NULL,  
  password VARCHAR(255) NOT NULL,  
  role VARCHAR(20) NOT NULL,  
  name VARCHAR(100),  
  department VARCHAR(100),  
  email VARCHAR(150) NOT NULL  
);
```

```
CREATE TABLE equipment (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100),  
  type VARCHAR(50),  
  status VARCHAR(20) DEFAULT 'available',  
  CONSTRAINT uq_equipment_name_type UNIQUE (name, type)  
);
```

```
CREATE TABLE allocations (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  equipment_id BIGINT NOT NULL,  
  user_id BIGINT NOT NULL,  
  allocated_at DATETIME NOT NULL,  
  expected_return DATETIME NOT NULL,  
  returned_at DATETIME,  
  equipment_condition VARCHAR(50),  
  reminder_sent BOOLEAN DEFAULT FALSE NOT NULL,  
  FOREIGN KEY (equipment_id) REFERENCES equipment(id),  
  FOREIGN KEY (user_id) REFERENCES users(id),  
  CONSTRAINT uq_active_allocation UNIQUE (equipment_id, returned_at)  
);
```

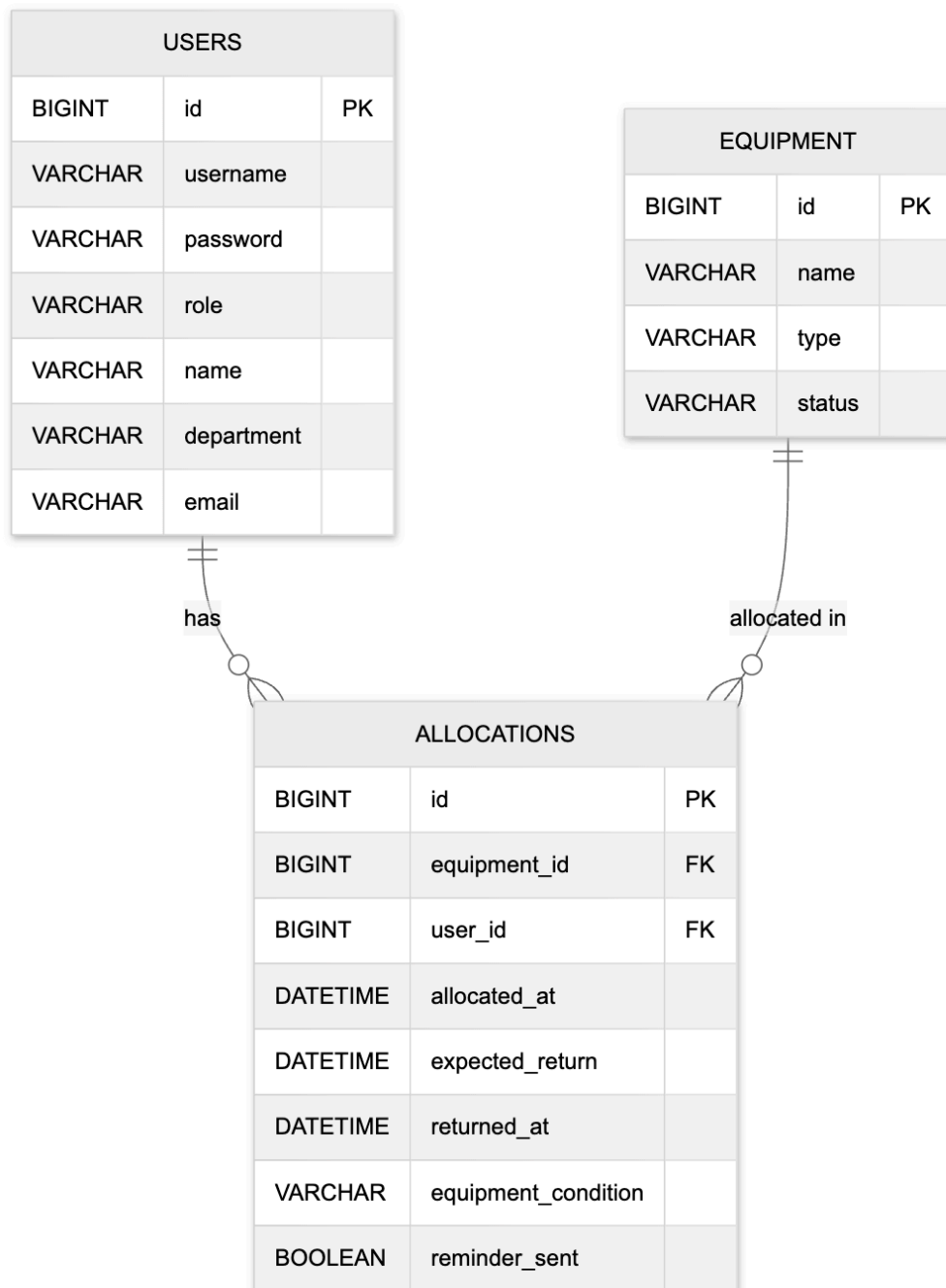
Insert Queries:

```
INSERT INTO users (id, username, password, role, name, department, email) VALUES
(1, 'admin', 'Admin@123', 'admin', 'Naresh Nagula', 'IT', 'naresh996463@gmail.com'),
(2, 'Hardhik', 'Hardhik@123', 'user', 'Hardhik', 'IT', 'naresh119935@gmail.com'),
(3, 'reception1', 'Recep@123', 'reception', 'Reception Staff', 'Front Desk',
'reception@example.com'),
(4, 'inventory1', 'Inv@123', 'inventory', 'Inventory Staff', 'IT', 'nareshnagula61@gmail.com'),
(5, 'maint1', 'Maint@123', 'maintenance', 'Maintenance Staff', 'Facilities',
'maintenance@example.com'),
(6, 'Rohith', 'Rohith@123', 'user', 'Rohith', 'IT', 'rohit123@gmail.com'),
(7, 'Kohli', 'Kohli@123', 'user', 'kohli', 'IT', 'kohli12@gmail.com'),
(8, 'Mahesh', 'Mahesh@123', 'user', 'mahesh', 'IT', 'mahesh2@gmail.com');
```

```
INSERT INTO equipment (id, name, type, status) VALUES
(1, 'hp laptop', 'Laptop', 'allocated'),
(2, 'apple laptop', 'Laptop', 'allocated'),
(3, 'Asus laptop', 'Laptop', 'allocated'),
(4, 'Apple laptop', 'Projector', 'allocated'),
(5, 'Mi', 'Projector', 'allocated'),
(6, 'Hp', 'Projector', 'damaged'),
(7, 'Dell', 'Laptop', 'available'),
(8, 'Intex', 'Laptop', 'available');
```

```
INSERT INTO allocations (id, equipment_id, user_id, allocated_at, expected_return, returned_at,
equipment_condition, reminder_sent) VALUES
(1, 1, 2, '2025-11-19 17:19:27', '2025-11-20 18:00:00', NULL, NULL, 1),
(2, 2, 6, '2025-11-19 17:27:17', '2025-11-20 18:00:00', NULL, NULL, 1),
(3, 3, 7, '2025-11-19 17:30:58', '2025-11-20 18:00:00', NULL, NULL, 1),
(4, 4, 8, '2025-11-19 17:39:59', '2025-11-20 18:00:00', NULL, NULL, 1),
(5, 5, 9, '2025-11-19 18:14:38', '2025-11-20 18:00:00', NULL, NULL, 1),
(6, 6, 10, '2025-11-19 18:19:40', '2025-11-20 18:00:00', '2025-11-19 22:42:59', 'Damaged', 0),
(7, 7, 11, '2025-11-19 18:58:12', '2025-11-20 06:42:23', NULL, 'Good', 1),
(8, 8, 10, '2025-11-19 21:14:27', '2025-11-20 18:00:00', '2025-11-19 22:15:35', 'Good', 0);
```

ERD DIAGRAM



9. Security

Authentication: JWT-based tokens for secure, stateless user authentication.

Authorization: Role-based access control.

1. Admin

Responsibilities: Full control — manage users, equipment, allocation, returns, and mark equipment as damaged.

Flow:

User Management:

- Admin creates/updates/deletes users via `/api/users/*` endpoints.
- DB is updated immediately.

Equipment Management:

- Admin adds new equipment or updates details via `/api/equipment/*` endpoints.
- Status changes (e.g., damaged) are updated in DB.

Equipment Allocation/Return:

- Admin can allocate or return equipment via `/api/equipment/allocate` or `/return`.

Allocation/return triggers Kafka events:

- Notify Inventory Staff (availability)
- Notify Maintenance Staff if returned item is damaged

Mark Damaged:

- Admin can mark damaged equipment using `/mark-damaged/:id`.
- Publishes `EquipmentDamagedEvent` to Kafka → Maintenance Staff Actor handles it asynchronously.
- Summary: Admin is full control, performs actions synchronously in DB, events are sent asynchronously for notifications.

2. Reception Staff

Responsibilities: Handle day-to-day allocation and return of equipment.

Flow:**Equipment Allocation:**

Receives allocation request → /allocate API called → DB updated.
Event published to Kafka → Inventory notified of decreased availability.

Equipment Return:

- Receives returned item → checks for damage:
- If OK → /return API called → DB updated → Inventory notified.
- If damaged → /mark-damaged/:id API called → DB updated → Maintenance notified via Kafka.

View Status:

- /status/:userId → view equipment allocated to specific user (read-only).
- Summary: Reception staff handles synchronous allocation/return and triggers asynchronous events for Inventory & Maintenance.

3. Inventory Staff

Responsibilities: Monitor inventory and availability.

Flow:**Receive Notifications:**

- Kafka consumer listens for allocation/return events → updates internal inventory view.
- Damaged items are flagged as unavailable.

View Equipment & Status:

- /equipment/list → read-only list of equipment
- /status/:userId → read-only allocation status
- Summary: Inventory staff do not change DB, only monitor and plan inventory based on asynchronous event-driven updates.

4. Maintenance Staff

Responsibilities: Handle damaged equipment, ensure repair and readiness for allocation.

Flow:

Receive Alerts:

Kafka consumer listens for EquipmentDamagedEvent.
Maintenance staff inspects damaged items.

Repair/Update Status:

- After repair, update equipment status via API → "ready" or "repaired".
- Event published to Kafka → Inventory notified that equipment is now available.

View Allocation Status:

- /status/:userId → read-only for planning repairs.
- Summary: Maintenance staff react to asynchronous events, update status after servicing, ensuring inventory accuracy.

End-to-End Flow Example:

Allocation → Return → Damaged → Repair → Availability

1. Admin adds new equipment.
2. Reception staff allocates equipment to Employee A → DB updated, Kafka event → Inventory notified.
3. Employee returns equipment damaged → Reception calls markDamagedEquipment → DB updated, Kafka event → Maintenance notified.
4. Maintenance staff repairs laptop → updates status to "ready" → Kafka event → Inventory notified.
5. Inventory now sees equipment available → can be allocated again.

10. Key Design Decisions

- **Microservice Architecture:** Enables independent scaling of API, Notification Service, and database.
- **Kafka Messaging:** Ensures reliable, asynchronous notifications.
- **Akka Actors:** Efficiently handle scheduled and real-time notifications.
- **Dockerization:** Guarantees consistent deployment across environments.
- **Database Normalization:** Prevents duplication of equipment and employee data.
- **REST API:** Standardized communication between frontend and backend.

11. Conclusion

The proposed architecture provides a **scalable, reliable, and automated solution** for corporate equipment management. It ensures:

- Timely allocation and return of equipment.
- Automated notifications to employees and internal teams.
- Accurate tracking of inventory and maintenance needs.
- Maintainable and extensible system design for future expansion.